

Research paper

Quantum optimisation for supply chain: QUBO formulations and QAOA solutions for facility location and load balancing

Luis A. Moncayo-Martínez ^{a,b,*}, Naihui He ^b

^a Department of Industrial Engineering and Operations, Instituto Tecnológico Autónomo de México (ITAM), Rio Hondo #1, Mexico City, 01080, Mexico

^b Department of Engineering, Faculty of Environment, Science and Economy, University of Exeter, Exeter, EX4 4QJ, UK

ARTICLE INFO

Keywords:

Quantum optimisation
Supply chain
Facility location
Line balancing
PennyLane
Qiskit

ABSTRACT

Quantum computing is emerging as a promising paradigm for solving combinatorial optimisation problems that are intractable for classical methods. Among the most prominent quantum algorithms is the Quantum Approximate Optimisation Algorithm (QAOA), which has demonstrated potential in addressing NP-hard problems using shallow circuits compatible with Noisy Intermediate-Scale Quantum (NISQ) devices. While QAOA has been applied to logistics and scheduling tasks, foundational supply chain problems such as Facility Location (FL) and Load Balancing (LB) remain largely unexplored.

This paper addresses this gap by formulating FL and LB as Quadratic Unconstrained Binary Optimisation (QUBO) models and solving them using QAOA under both noiseless and hardware-calibrated noisy conditions. Two encoding methods are compared: the conventional slack-variable approach and the unbalanced penalisation technique. Results show that unbalanced penalisation consistently outperforms the slack-variable method by achieving higher feasibility and optimality rates and by avoiding the qubit inflation that limits slack-based encodings. FL instances, modelled as minimum dominating sets, exhibit strong performance in both noiseless and noisy simulations, whereas LB instances suffer from depth-induced noise due to their denser constraints. Furthermore, we outline the QUBO-to-Ising transformation process, with open-source implementations in PennyLane and Qiskit. To the best of our knowledge, this work represents the first application of QAOA to FL and LB, bridging the gap between quantum algorithm design and practical supply chain optimisation.

1. Introduction

Quantum computing is an emerging paradigm that offers a promising approach for solving complex optimisation problems by exploiting quantum parallelism and entanglement. However, current hardware limitations—such as short coherence times, limited qubit counts, and high error rates—still constrain its scalability and practical implementation. To address these limitations, Variational Quantum Algorithms (VQAs) have become widely studied, particularly in the field of optimisation [1]. VQAs combine quantum state preparation with classical optimisation loops, making them well suited for current Noisy Intermediate-Scale Quantum (NISQ) devices [2]. By requiring shallow circuits and fewer qubits compared with fully fault-tolerant algorithms, VQAs enable practical experimentation on existing hardware. Among them, the Quantum Approximate Optimisation Algorithm (QAOA) stands out as a leading approach for solving discrete combinatorial problems, offering a structured framework to approximate optimal solutions through alternating applications of cost and mixer Hamiltonians.

The applicability of QAOA to industrial optimisation has attracted growing interest across logistics, manufacturing, energy, and

telecommunications. It has been applied to problems such as job-shop scheduling, last-mile delivery, cargo loading, and vehicle routing, all of which are NP-hard and central to operational efficiency [3–5]. In power systems, QAOA has been used for unit commitment and load dispatch [6], while in telecommunications, it has optimised antenna placement and network coverage [7]. These applications demonstrate the algorithm's flexibility when paired with suitable Quadratic Unconstrained Binary Optimisation (QUBO) formulations. Although most implementations are currently limited to small-scale or simulated instances due to hardware constraints, QAOA has shown its capability to produce high-quality feasible solutions using shallow quantum circuits, positioning it as a viable component in hybrid quantum–classical workflows.

In the context of supply chain optimisation, QAOA has been explored for solving complex combinatorial problems such as fleet routing, inventory control, cargo loading, order picking, and network design [3,4,8]. In particular, it has shown encouraging results in vehicle routing problems, where feasible routes are derived from QUBO formulations under capacity and distance constraints, and in inventory management, where it helps balance holding and shortage costs. Despite this

* Corresponding author.

<https://doi.org/10.1016/j.rineng.2025.108373>

Received 22 August 2025; Received in revised form 15 November 2025; Accepted 20 November 2025

Available online 27 November 2025

2590-1230/© 2025 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY-NC license (<http://creativecommons.org/licenses/by-nc/4.0/>).

progress, fundamental challenges in supply-chain design and production planning remain largely unexplored. To the best of current knowledge, no prior studies have addressed the Facility Location (FL) and Load Balancing (LB) problems using QAOA [3,4,8]. These problems are foundational to manufacturing and logistics: FL determines the placement of service units to minimise cost or service time, whereas LB distributes tasks among workstations to reduce idle time and increase throughput. Both are NP-hard and critical for cost reduction and resource utilisation, yet they have not been investigated under a QAOA framework.

This paper addresses this gap by encoding both the FL and LB problems as QUBO models and solving them using QAOA in two quantum software frameworks widely used in research. In PennyLane [9], both problems are implemented under noise-free and non-optimised conditions to verify the correctness of the QUBO and Ising encodings, analyse the corresponding energy landscapes, and evaluate circuit depth without classical parameter tuning. In Qiskit [10], the same models are executed using the hardware-calibrated noise model of the `ibm_brisbane` backend, with the variational parameters optimised through the COBYLA algorithm to minimise the energy of the cost Hamiltonian. This dual implementation enables both theoretical validation and performance benchmarking across ideal and hardware-representative environments.

This work compares the conventional slack-variable QUBO encoding with the unbalanced-penalisation method, which eliminates auxiliary (slack) variables, reduces circuit width, and improves the probability of sampling feasible solutions. The results show that using the unbalanced-penalisation method allows QAOA to find a higher number of optimal solutions, consistent with the findings in [11].

Recent comparative studies of quantum frameworks reinforce the importance of evaluating both PennyLane and Qiskit. The analysis in [12] shows that PennyLane offers device-agnostic flexibility suited to research in VQAs and quantum machine learning, while Qiskit provides an environment more aligned with education and hardware-oriented experimentation. Similar conclusions are drawn in [13]. These findings justify the use of both frameworks in this study, along with the unbalanced-penalisation method, to improve QUBO compactness and computational efficiency.

A second objective of this work is to provide a transparent and reproducible methodology for transforming constrained optimisation problems into QUBO and Ising Hamiltonians suitable for QAOA and other VQAs. This contribution complements existing computational packages such as [14–16], which allow users to input constraint equations but do not expose the underlying process of Hamiltonian construction. Conversely, several tutorials describe how to transform an optimisation problem into a QUBO model [17–19], but they lack computational implementation. In contrast, this work provides both the theoretical details required to compute the Ising model and the practical guidance to implement the corresponding quantum circuit. The complete project is publicly available at https://github.com/LuisMoncayo/Quantum_Opt_Supply_Chain.

Finally, this work builds upon foundational advances in hybrid quantum computation. VQAs provide a general framework for solving optimisation problems using parameterised quantum circuits whose expressibility grows with depth but also increases noise. QAOA, a VQA tailored to combinatorial optimisation, aims to remain shallow enough to be robust while still expressive enough to represent high-quality solutions [20]. Experiments on superconducting hardware have validated that QAOA improves solution quality as circuit depth increases [21], and the approach has been extended to nonplanar graphs using machine-learning-based noise mitigation on up to forty qubits [22]. More recently, QAOA has been positioned within the broader quantum machine-learning paradigm [13]. Collectively, these works provide the theoretical and experimental context for the present study, which applies both noise-free and noise-aware QAOA implementations to combinatorial supply-chain optimisation in facility location and load balancing.

2. Literature review

Variational Quantum Algorithms (VQAs) constitute a class of hybrid quantum-classical algorithms explicitly designed for Noisy Intermediate-Scale Quantum devices. These algorithms integrate quantum state preparation with classical optimisation to address challenging combinatorial optimisation problems [23,24]. Among them, the Quantum Approximate Optimisation Algorithm (QAOA) stands out as a widely studied VQA, particularly suited for solving discrete optimisation problems using shallow quantum circuits [25]. QAOA is particularly promising for tackling combinatorial problems that are computationally demanding for classical solvers. For example, in the MaxCut variant known as Max E3LIN2, QAOA has been shown to outperform any known polynomial-time classical algorithm in terms of approximation ratio [26].

Recent surveys highlight growing interest in applying VQAs to industrial optimisation problems [1,27], including antenna network power consumption [7], power systems [6], and finance [28]. In particular, QAOA has been employed in various supply chain applications such as fleet optimisation, cargo loading, inventory control, scheduling, vehicle routing, and network design [4,8]. Within production management, it has also been applied to job shop scheduling, operations planning, order picking, robotic movement planning, and last-mile delivery [3,5].

Despite this progress, limited attention has been given to fundamental combinatorial problems such as facility location (FL), which are central to supply chain design. In this work, the FL problem is modelled as a minimum dominating set, where the objective is to open the fewest number of service facilities in a set of locations such that every location is either selected or adjacent to one of the selected facilities. In contrast, related studies such as [29] formulate the FL problem to minimise the customer servicing costs and facility opening costs, using slack-variable encoding and an ad-hoc VQA that preserves feasibility. The same problem is presented in [30], where D-Wave annealers are used with a slack-based QUBO model. In [31], the FL problem is modelled as a maximal covering location problem, aiming to maximise demand coverage given a fixed number of facilities; the authors encode small instances into QUBO using equivalent penalty constraints and solve them with D-Wave annealers.

Regarding the line balancing (LB) problem, to the best of our knowledge, this study is the first to solve it using QAOA with a QUBO encoding based on the unbalanced penalisation method, as shown in [32]. In [33], D-Wave quantum annealers are applied to a robotic assembly line involving four tasks and two cells, minimising the cost of assigning robotic arms to tasks. A quasi-quantum approach is also explored in [34] for disassembly line balancing, using qubit-based representations to explore feasible task allocations. However, none of these approaches implement QAOA or investigate alternative QUBO encodings beyond standard slack variables.

In addition to addressing the FL and LB problems using QAOA, this work introduces a comparative study of two QUBO encoding strategies: the standard slack-variable method and the unbalanced penalisation approach. It provides a detailed formulation of both problems in QUBO form and their subsequent transformation into the Ising model - the Hamiltonian required for any VQA. The ability to encode binary optimisation problems in both QUBO and Ising form is essential for implementing VQAs and is increasingly recognised as a fundamental skill for researchers and practitioners [14]. Several tools have been developed to support this process, including Qubo.jl [14], which enables QUBO modelling in Julia; an encoding framework with custom penalty functions [15]; and PyQUBO [16], a Python-based implementation that uses the slack-variable method. This work complements these tools by offering a practical and transparent guide for encoding supply chain optimisation problems in a quantum-ready form, enabling users to gain insight into the problem's structure rather than merely inputting constraints.

While theoretical developments in QUBO modelling continue to advance, a recurring limitation is the lack of computational implemen-

tation. For example, [17] proposes a method to approximate objective functions as QUBO via polynomial expansions, but does not provide executable code or integration with quantum workflows. Similarly, [18] presents a comprehensive tutorial on QUBO modelling, and [18] introduces the unbalanced penalisation technique, yet neither study includes numerical experiments or software implementations. In [35], supply chain problems are reformulated as QUBO models, but no classical or quantum solution methods are proposed. To address this gap, the present study offers complete, open-source Python implementations for encoding and solving the FL and LB problems using QAOA, available at https://github.com/LuisMoncayo/Quantum_Opt_Supply_Chain.

In summary, this work contributes to the growing literature by formulating two fundamental supply chain problems - facility location and load balancing - as QUBO models and solving them using QAOA. It presents a comparative analysis of slack-based and unbalanced penalisation encoding methods, both theoretically and computationally. To the best of our knowledge, this is the first study to apply QAOA to the load balancing problem and to deliver reproducible, end-to-end implementations that bridge the gap between quantum optimisation theory and practical applications in industrial contexts.

3. Problems definition

3.1. Facility location problem

The Facility Location (FL) problem, in its variant known as the Minimum Dominating Set, is an NP-hard combinatorial optimisation problem. In this formulation, a set of locations (j) must be served by the smallest possible number of facilities. Each location can potentially have a facility installed; therefore, $x_j = 1$ if a facility is opened at location j , and $x_j = 0$ otherwise. Furthermore, if no facility is placed at a given location, that location must be adjacent to one or more where a facility is installed. The set of locations adjacent to j is denoted by $J_j = \{j' \mid j' \text{ is adjacent to } j\}$, noting that j is included in its own neighbourhood. Mathematically, Eq. 1 represents the FL problem as a binary programming model.

$$\begin{aligned} \min \quad & \sum_j x_j \\ \text{s.t.} \quad & \sum_{j' \in J_j} x_{j'} \geq 1, \quad \forall J_j, \\ & x_j \in \{0, 1\}, \quad \forall j. \end{aligned} \quad (1)$$

For example, consider a simple case with three locations ($j = 1, 2, 3$) and their neighbourhoods $J_1 = \{1, 2\}$, $J_2 = \{1, 2, 3\}$, and $J_3 = \{2, 3\}$. In this case, Eq. 1 becomes $\min x_1 + x_2 + x_3$ subject to $x_1 + x_2 \geq 1$, $x_1 + x_2 + x_3 \geq 1$, $x_2 + x_3 \geq 1$, and $x_1, x_2, x_3 \in \{0, 1\}$. If facilities are opened at locations 1 and 3 ($x_1 = x_3 = 1, x_2 = 0$), all locations are covered, since location 2 is adjacent to both 1 and 3; thus, $\{1, 3\}$ forms a feasible dominating set for this instance. Alternatively, opening a single facility at location 2 ($x_2 = 1, x_1 = x_3 = 0$) also satisfies all constraints, since locations 1 and 3 are adjacent to 2. Therefore, $\{2\}$ is another feasible dominating set.

3.2. Load balancing problem

The Load Balancing (LB) problem is a classical NP-hard optimisation problem in operations research. In this problem, a set of tasks (r) must be assigned to a set of workstations (k), subject to a cycle time (C) that represents the maximum allowable processing time at each station. Each task r has a known deterministic processing time (t_r). The binary decision variables are defined as $w_{rk} = 1$ if task r is assigned to station k , and $w_{rk} = 0$ otherwise; and $y_k = 1$ if station k is opened, and $y_k = 0$ otherwise.

The objective is to assign all tasks to the minimum number of stations (Eq. 2a), ensuring that the total processing time of the tasks assigned to each station does not exceed the cycle time C (Eq. 2b), each task r is

assigned to exactly one station k (Eq. 2c), and all precedence relations among task pairs (r, r') are satisfied. For example, if task r is assigned to station k , its successor task r' must be assigned to the same k or to a subsequent station $k + 1, k + 2, \dots$.

$$\min \quad \sum_k y_k \quad (2a)$$

$$\text{s.t.} \quad \sum_r t_r w_{rk} \leq C y_k, \quad \forall k, \quad (2b)$$

$$\sum_k w_{rk} = 1, \quad \forall r, \quad (2c)$$

$$\sum_k k \cdot w_{rk} \leq \sum_k k \cdot w_{r'k}, \quad \forall (r, r'), \quad (2d)$$

$$y_k, w_{rk} \in \{0, 1\}, \quad \forall r, k. \quad (2e)$$

In this formulation, the number of open stations could be computed by divided the total processing time and the cycle time; $k = 1, 2, \dots, \lceil \sum_r t_r / C \rceil$; thus, $y_1, y_2 = \dots = y_{\lceil \sum_r t_r / C \rceil} = 1$. For example suppose the problem with three tasks $r = 1, 2, 3$, the processing times $t_r = \{2, 3, 2\}$, the cycle time $C = 4$; thus, the maximum number of open stations is $\lceil (2 + 3 + 2) / 4 \rceil = 2$, and the precedence relations are (1,2) and (2,3), meaning that task 1 must be completed before task 2, and task 2 before task 3. The model is $\min y_1 + y_2 = 2$ subject to $2w_{11} + 3w_{21} + 2w_{31} \leq 4$, $2w_{12} + 3w_{22} + 2w_{32} \leq 4$, $w_{11} + w_{12} + w_{13} = 1$, $w_{21} + w_{22} + w_{23} = 1$, $w_{31} + w_{32} + w_{33} = 1$, $1 \cdot w_{11} + 2 \cdot w_{12} \leq 1 \cdot w_{21} + 2 \cdot w_{22}$, and $1 \cdot w_{21} + 2 \cdot w_{22} \leq 1 \cdot w_{31} + 2 \cdot w_{32}$. The optimal solution opens two stations ($y_1 = y_2 = 1$) with assignments $w_{11} = w_{21} = 1$ and $w_{32} = 1$. Tasks 1 and 2 are performed at station 1, while task 3 is processed at station 2, satisfying both the cycle time and precedence constraints.

3.3. General QUBO and ising formulation

The QUBO formulation serves as an intermediate step between the classical binary optimisation model and its quantum implementation. By embedding all constraints as quadratic penalty terms, the original constrained problem is transformed into an unconstrained quadratic form that can be directly mapped to an Ising Hamiltonian. The Ising representation, expressed in terms of spin variables $z_i \in \{-1, +1\}$, is essential for implementation on circuit-based algorithms such as QAOA.

A QUBO problem can be expressed as shown in Eq. 3, assuming that the coefficient matrix Q is upper-triangular so that $i < j$ holds.

$$\min_{\mathbf{x} \in \{0,1\}^n} f(\mathbf{x}) = \mathbf{x}^T Q \mathbf{x} + \text{offset} = \sum_{i=1}^n q_{ii} x_i + \sum_{i=1}^{n-1} \sum_{j=i+1}^n q_{ij} x_i x_j + \text{offset}, \quad (3)$$

$\mathbf{x} = [x_1, x_2, \dots, x_n]^T$ is the vector of binary decision variables and $Q \in \mathbb{R}^{n \times n}$ is the matrix of coefficients. Each diagonal element q_{ii} represents a linear contribution, while each off-diagonal element q_{ij} encodes a quadratic interaction between variables x_i and x_j .

A binary programming model, such as those in Eq. 1 and Eq. 2, can be reformulated as a QUBO by embedding its constraints into the objective function through quadratic penalty terms (see § 3.4). After algebraic expansion and simplification, the formulation preserves the quadratic structure of Eq. 3 and can be directly mapped to the Ising Hamiltonian:

$$H(\mathbf{z}) = \sum_i h_i z_i + \sum_{i < j} J_{ij} z_i z_j + \text{offset}_H,$$

where $z_i \in \{-1, +1\}$ are spin variables, h_i represents the local magnetic field acting on spin z_i , and J_{ij} denotes the coupling strength between spins z_i and z_j .

The transformation from the QUBO to the Ising form is obtained by substituting $x_i = \frac{1+z_i}{2}$ into Eq. 3. After expanding and rearranging terms,

the coefficients are obtained as in Eq. 4.

$$H(\mathbf{z}) = \sum_i \left(-\frac{1}{2} q_{ii} - \frac{1}{4} \sum_{j \neq i} q_{ij} \right) z_i + \sum_{i < j} \frac{1}{4} q_{ij} z_i z_j + \left(\frac{1}{2} \sum_i q_{ii} + \frac{1}{4} \sum_{i < j} q_{ij} + \text{offset} \right) \quad (4)$$

3.4. Encoding methods to generate the QUBO

The first step in applying QAOA to a binary combinatorial problem is to encode it as a QUBO model. This transformation converts the constrained formulation into an unconstrained quadratic form, where the penalty coefficients control the balance between minimising the original objective and enforcing constraint satisfaction.

3.4.1. The slack-variable method

The slack-variable method (SVM) transforms all constraints into equalities and incorporates them into the objective function through penalty terms weighted by a coefficient λ . Inequalities of the form $\leq$ are converted into equalities by adding a positive integer slack variable $+s$, while those of the form $\geq$ are converted by subtracting a positive integer surplus variable $-s$; in this context, both are collectively referred to as slack variables.

Each integer slack variable must be converted into its binary equivalent. This is achieved using $s = \sum_{a=0}^b 2^a x_a$ when s has a finite upper bound u , where b is the smallest integer satisfying $u \leq 2^{b+1} - 1$ [36]. It is important to note that no additional variables are required for constraints expressed as equalities.

Consider a simple illustrative example with three decision variables ($n_1 = 3$): minimise $6x_1 + x_2 + 4x_3$ subject to $x_1 + x_2 \leq 1$, $x_1 + x_2 + x_3 \geq 2$, and $x_1, x_2, x_3 \in \{0, 1\}$. The inequalities are transformed into equalities as $x_1 + x_2 + s_1 = 1$ and $x_1 + x_2 + x_3 - s_2 = 2$, where $s_1, s_2 \geq 0$ are integers. Each slack variable is then represented in binary form. The upper bound u of each slack variable equals the right-hand side of its corresponding constraint, giving $u_1 = 1$ and $u_2 = 2$.

For s_1 , the smallest integer b satisfying $u_1 \leq 2^{b+1} - 1$ is found sequentially: when $b = 0$, $1 \leq 1$; thus $b = 0$ and $s_1 = 2^0 x_4$. The indexing x_{a+n_1+1} ensures that variable numbering continues after the original decision variables; thus, the number of variables after adding the slacks to the first constraint is $n_2 = 4$. For the second constraint, where $u_2 = 2$, we find $b = 0 \Rightarrow 2 \not\leq 1$ and $b = 1 \Rightarrow 2 \leq 3$; hence $b = 1$, and $s_2 = 2^0 x_5 + 2^1 x_6$ (indexing x_{a+n_2+1}). Substituting back, the constraint becomes $x_1 + x_2 + x_3 - x_5 - 2x_6 - 2$ due to the negative slack variable $-s_2$.

The resulting QUBO formulation is

$$\min_{\mathbf{x}} 6x_1 + 5x_2 + 2x_3 + \lambda(x_1 + x_2 + x_4 - 1)^2 + \lambda(x_1 + x_2 + x_3 - x_5 - 2x_6 - 2)^2$$

The linear terms are expressed as $x_j = x_j^2$ since $x_j \in \{0, 1\}$. After algebraic expansion and aggregation of coefficients, the resulting QUBO (Eq. 3) can be written as:

$$\min_{\mathbf{x} \in \{0,1\}^n} f(\mathbf{x}) = [x_1 \quad \dots \quad x_6] \begin{bmatrix} 10\lambda + 6 & 4\lambda & 2\lambda & 2\lambda & -2\lambda & -4\lambda \\ 0 & 10\lambda + 5 & 2\lambda & 2\lambda & -2\lambda & -4\lambda \\ 0 & 0 & 5\lambda + 2 & 0 & -2\lambda & -4\lambda \\ 0 & 0 & 0 & 3\lambda & 0 & 0 \\ 0 & 0 & 0 & 0 & 5\lambda & 4\lambda \\ 0 & 0 & 0 & 0 & 0 & 8\lambda \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{bmatrix} + 5\lambda$$

The parameter λ acts as a penalty coefficient that balances minimisation of the original objective function against enforcement of the penalised constraints. In practice, λ is typically chosen empirically through scaling or testing: if λ is too small, infeasible solutions may be obtained; if λ is too large, the penalty terms dominate the search and may cause numerical instability [19].

To obtain the cost Hamiltonian (Ising form) $H(\mathbf{z})$, the elements $q_{ij} \in Q$ are substituted into Eq. 4. For simplicity, let $\lambda = 1$; thus, the cost Hamiltonian required by QAOA is:

$$H(\mathbf{z}) = z_1 z_2 + 0.5 z_1 z_3 + 0.5 z_1 z_4 - 0.5 z_1 z_5 - z_1 z_6 + 0.5 z_2 z_3 + 0.5 z_2 z_4 - 0.5 z_2 z_5 - z_2 z_6 - 0.5 z_3 z_5 - z_3 z_6 + z_5 z_6 - 8.5 z_1 - 7.0 z_2 - 2.0 z_3 - 1.5 z_4 - 3.5 z_5 - 4.0 z_6 + 30.5$$

3.4.2. The unbalanced penalisation method

The unbalanced penalisation method (UPM) reformulates a constrained binary optimisation problem into a QUBO without introducing slack variables. Each constraint is directly embedded into the objective function, without first being converted into an equality, using linear and quadratic penalty terms weighted by coefficients λ_1 and λ_2 .

The QUBO models of the same illustrative example problem used in §3.4.1 using UPM is:

$$\min_{\mathbf{x}} 6x_1 + x_2 + 4x_3 - \lambda_1 [(x_1 + x_2 - 1) + (x_1 + x_2 + x_3 - 2)] + \lambda_2 [(x_1 + x_2 - 1)^2 + (x_1 + x_2 + x_3 - 2)^2]$$

After expansion and simplification, the compactly QUBO form can be written as:

$$f(\mathbf{x}) = [x_1 \quad x_2 \quad x_3] \begin{bmatrix} 8\lambda_2 + 6 - 2\lambda_1 & 4\lambda_2 & 4\lambda_2 \\ 0 & 8\lambda_2 + 1 - 2\lambda_1 & 4\lambda_2 \\ 0 & 0 & 4\lambda_2 + 4 - \lambda_1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + 5\lambda_1 + 10\lambda_2$$

The linear penalty term ($-\lambda_1$) guides the optimiser toward feasibility, while the quadratic term ($+\lambda_2$) penalises constraint violations more strongly, ensuring feasible solutions. The UPM eliminates slack variables, reducing qubit requirements and circuit complexity, but the added linear term can produce denser coupling matrices Q and deeper QAOA circuits [11,19].

The cost Hamiltonian $H(\mathbf{z})$ when $\lambda_1 = \lambda_2 = 1$ is computed by substituting the elements of the matrix Q in Eq. 4. Therefore,

$$H(\mathbf{z}) = z_1 z_2 + z_1 z_3 + z_2 z_3 - 8z_1 - 4.5z_2 - 3.5z_3 + 31.$$

3.5. Transformation of the facility location problem

The number of variables (n) in the FLP (Eq. 1) corresponds to the number of locations. To obtain the QUBO model under the SVM approach (§ 3.4.1), the upper bound determining the number of additional slack variables is defined as $u_j = |J_j| - 1$ for each location j . Each slack variable is then expressed as $s_j = \sum_{a_j=0}^{b_j} 2^{a_j} x_{a_j}$, where the smallest integer b_j satisfies $|J_j| - 1 \leq 2^{b_j+1} - 1$. The resulting QUBO formulation for the FLP under the SVM approach is given by Eq. 5.

$$\min_{\mathbf{x} \in \{0,1\}^n} f_F^S(\mathbf{x}) = \sum_j x_j + \lambda \sum_j \left(\sum_{j' \in J_j} x_{j'} - \sum_{a_j=0}^{b_j} 2^{a_j} x_{a_j+n_j+1} - 1 \right)^2 \quad (5)$$

On the other hand, when the UPM is used to encoding the problem, the resulting QUBO model is computed by Eq. 6.

$$\min_{\mathbf{x} \in \{0,1\}^n} f_F^U(\mathbf{x}) = \sum_j x_j - \lambda_1 \sum_j \left(\sum_{j' \in J_j} x_{j'} - 1 \right) + \lambda_2 \sum_j \left(\sum_{j' \in J_j} x_{j'} - 1 \right)^2 \quad (6)$$

After expansion and term aggregation, each coefficient $q_{ij} x_i x_j$ is placed in the corresponding position of the matrix Q in Eq. 3. Then, the cost Hamiltonian by substituting q_{ij} in Eq. 4.

3.6. Transformation of the load balancing problem

The total number of binary variables w_{rk} , denoted as W , in the LB problem (Eq. 2, see § 3.2), is obtained by multiplying the total number of tasks by the total number of stations. The number of binary variables y_k , associated with the open facilities, is determined by dividing the total processing time of all tasks ($\sum_r t_r$) by the cycle time (C). Therefore, the minimum number of open facilities is a known value.

To construct the QUBO model using the SVM, the upper bound of the cycle time constraints (Eq. 2b) is defined as $u_k = C - \min_r \{t_r\}$, representing the worst-case scenario in which only the task with the shortest processing time is assigned to a station. The corresponding slack variables are expressed as $s_k = \sum_{a_k=0}^{b_k} 2^{a_k} x_{a_k+W_k+1}$, where $C - \min_r \{t_r\} \leq 2^{b_k+1} - 1$. For $k = 1$, $W_1 = W$, and each subsequent W_k is updated recursively as $W_k = W_{k-1} + (\text{number of slack variables added in } s_{k-1})$.

For the task assignment constraints (Eq. 2c), no additional variables are required. In the case of the precedence constraints (Eq. 2d), the right-hand side is ≤ 0 ; hence $u_{(r,r')} = 0$, and the corresponding additional binary variables are defined as $s_{(r,r')} = \sum_{a_{(r,r')}=0}^{b_{(r,r')}} 2^{a_{(r,r')}} x_{a_{(r,r')}+W_{(r,r')}+1}$, where $W_{(r,r')}$ represents the cumulative number of binary variables introduced in the other constraints. Thus, for the precedence constraints, only one additional binary variable is needed since $0 \leq 2^{b_{(r,r')}+1} - 1$. The QUBO model using the ASV is presented in Eq. 7

$$\begin{aligned} \min_{\mathbf{x} \in \{0,1\}^n} f_B^S(\mathbf{x}) = & + \lambda \sum_k \left(\sum_r t_r w_{rk} + \sum_{a_k=0}^{b_k} 2^{a_k} x_{a_k+W_k+1} - C \right)^2 \\ & + \lambda \sum_r \left(\sum_k w_{rk} - 1 \right)^2 \\ & + \lambda \sum_{(r,r')} \left(\sum_k k \cdot w_{rk} - \sum_k k \cdot w_{r'k} \right. \\ & \left. + \sum_{a_{(r,r')}=0}^{b_{(r,r')}} 2^{a_{(r,r')}} x_{a_{(r,r')}+W_{(r,r')}+1} \right)^2 \end{aligned} \quad (7)$$

Eq. 8 presents the QUBO formulation of the LBP using UPM (see § 3.4.2). The first group of terms enforces the cycle time limits for each workstation, the second ensures that each task is assigned to exactly one station, and the third preserves the required precedence relationships among tasks. This formulation reduces the number of binary variables compared to the SVM, leading to a smaller qubit count and lower circuit complexity in quantum implementations.

$$\begin{aligned} \min_{\mathbf{x} \in \{0,1\}^n} f_B^U(\mathbf{x}) = & - \lambda_1 \sum_k \left(\sum_r t_r w_{rk} - C y_k \right) + \lambda_2 \sum_k \left(\sum_r t_r w_{rk} - C y_k \right)^2 \\ & - \lambda_1 \sum_r \left(\sum_k w_{rk} - 1 \right) + \lambda_2 \sum_r \left(\sum_k w_{rk} - 1 \right)^2 \\ & - \lambda_1 \sum_{(r,r')} \left(\sum_k k \cdot w_{rk} - \sum_k k \cdot w_{r'k} \right) \\ & + \lambda_2 \sum_{(r,r')} \left(\sum_k k \cdot w_{rk} - \sum_k k \cdot w_{r'k} \right)^2 \end{aligned} \quad (8)$$

Both the SVM and UPM formulations yield quadratic unconstrained binary optimisation models that can be expressed in the canonical QUBO form (see Eq. 3). By applying the standard transformation $x_i = (1 + z_i)/2$, these QUBO formulations are mapped into their equivalent Ising Hamiltonians (see Eq. 4). This transformation enables the direct implementation of the LB and FL problems on quantum hardware through the QAOA.

4. Materials and methods

4.1. Quantum approximate optimisation algorithm

The Quantum Approximate Optimisation Algorithm (QAOA), proposed by Farhi et al. [37], is a hybrid quantum-classical variational framework used to solve combinatorial optimisation problems expressible in the Ising form. The algorithm is particularly suitable for Noisy Intermediate-Scale Quantum (NISQ) devices, as it requires only shallow quantum circuits and offloads variational parameter tuning to classical optimisation routines.

QAOA is inspired by the adiabatic theorem of quantum mechanics, which asserts that a system initially prepared in the ground state of a Hamiltonian will remain in that state provided the Hamiltonian evolves sufficiently slowly. Rather than implementing continuous evolution, QAOA discretises the process into a finite sequence of alternating unitary operations, thereby constructing a circuit-based variational ansatz.

The QAOA ansatz consists of $l = 1, \dots, p$ layers, each comprising alternating cost unitary operator $U_C(\gamma_l)$ and mixer unitary operator $U_M(\beta_l)$ applied at layer l , as shown in Eq. 9.

$$\begin{aligned} U_C(\gamma_l) &= \prod_{q=1}^n e^{-i\gamma_l H_C} = \left(\prod_{q=1}^n e^{-i\gamma_l H_{q,q'}} \right) \left(\prod_{q < q'} e^{-i\gamma_l J_{qq'} z_q z_{q'}} \right) \\ U_M(\beta_l) &= \prod_{q=1}^n e^{-i\beta_l H_M} = \prod_{q=1}^n e^{-i\beta_l X_q} \end{aligned} \quad (9)$$

where H_C is the cost Hamiltonian encoding the problem objective, and H_M is the mixer Hamiltonian and is not unique, but often it is taken as the sum of Pauli- X operators on each qubit $q = 1, \dots, n$ to explore the solution space and ensure ergodicity. The parameters γ_l and β_l serve as rotation angles controlling the evolution under the respective Hamiltonians in layer l . This procedure results in the variational state

$$|\psi(\vec{\gamma}, \vec{\beta})\rangle = \left(\prod_{l=1}^p U_C(\gamma_l) U_M(\beta_l) \right) |\psi_0\rangle \quad (10)$$

where $|\psi_0\rangle = H^{\otimes n} |0\rangle^{\otimes n}$ denotes the equal superposition state of n qubits, and also corresponds to the ground state of the mixer Hamiltonian $H_M = \sum_{q=1}^n X_q$. In the limit $p \rightarrow \infty$, QAOA approaches the continuous adiabatic evolution path.

The QAOA is inherently statistical due to the probabilistic nature of quantum measurement. The final variational state as described in Eq. 10 is a superposition over all computational basis states. Upon measurement, the quantum system collapses to a single bitstring or sample outcome $s = (s_1, \dots, s_q, \dots, s_n)$, $s_q \in \{0, 1\}$ with probability determined by the squared amplitude of that state as in Eq. 11

$$P(s) = \left| \langle s | \psi(\vec{\gamma}, \vec{\beta}) \rangle \right|^2 \quad (11)$$

where $|s\rangle = |s_1, \dots, s_q, \dots, s_n\rangle$ denotes the basis state associated with the sample s . By repeatedly measuring the quantum state, one obtains a set of samples that form an empirical distribution over the solution space, with the goal of concentrating probability mass on optimal or near-optimal solutions.

4.2. QAOA-based solution approach

In this work, two widely adopted quantum software frameworks, **PennyLane** and **Qiskit** [38], were employed to implement the proposed QAOA-based methodology.

Algorithm 1 summarises the complete workflow used to obtain both noiseless and noisy QAOA solutions for combinatorial optimisation problems, including the Facility Location (FL) and Load Balancing (LB) models. The workflow is organised into three main stages: initialisation, circuit preparation and measurement, and sample validation.

PennyLane [9] served as a high-level interface for constructing and simulating parameterised quantum circuits within the proposed

methodology. In this study, PennyLane operated in deterministic mode without gradient-based optimisation or noise modelling. It was employed to validate the structure of the cost and mixer Hamiltonians and to confirm the correct behaviour of the QAOA ansatz under ideal conditions.

Qiskit [39] provided the primary environment for executing the QAOA experiments under realistic noise conditions. The algorithm was implemented on the IBM hardware-calibrated backend `ibm_brisbane`, which incorporates device-specific calibration data such as gate and readout error rates. The `AerSimulator` module was configured with this noise model to emulate execution on a realistic NISQ device. Parameter optimisation for the variational angles γ and β was performed using the COBYLA optimiser, a derivative-free constrained method well suited to low-dimensional, noise-affected search spaces. This configuration enabled assessment of QAOA performance, stability, and reproducibility under both ideal and hardware-level noisy conditions.

Algorithm 1: QAOA-Based Solution Approach.

```

Input: A combinatorial optimisation model, e.g., FL or LB
Output: Optimal solution under noiseless and noisy conditions
1 Encode the QUBO (Eq. 3) by SVM (§3.4.1) or UPM (§3.4.2);
2 Formulate as an Ising model  $H(\mathbf{z})$  (Eq. 4);
  /* Initialisation */
3 Define the cost Hamiltonian  $H_C$  using Pauli-Z operators;
4 Define the mixer Hamiltonian  $H_M$  using Pauli-X operators;
  /* Circuit preparation */
5 Set the number of layers  $l = 1, \dots, p$  and shots  $\tau = 1, \dots, S$ ;
6 Set a data frame called Samples;
  /* Ideal (Noiseless) */
7 if PennyLane then
8   Set  $\gamma = [\gamma_1, \dots, \gamma_p]$  and  $\beta = [\beta_1, \dots, \beta_p]$ ;
9   for  $\tau = 1$  to  $S$  do
10    Initialise the quantum circuit  $|\psi_0\rangle = H^{\otimes n}|0\rangle^{\otimes n}$ ;
11    for  $l = 1$  to  $p$  do
12      Apply  $U_C(\gamma_l)$  and  $U_M(\beta_l)$  (Eq. 9);
13    /* Measurement */
14    Return  $|\psi(\gamma, \beta)\rangle$  (see Eq. 10);
15    Measure the probabilities  $P(s_\tau)$  (Eq. 11);
16    Samples  $\leftarrow s_\tau = (s_1, \dots, s_n)$ ,  $s_i \in \{0, 1\}$ ;
17  Go to line 24;
  /* Noisy Solution and Parameter Optimisation */
18 else if Qiskit then
19   Create the parametrised quantum circuit (ansatz);
20   Optimise  $\gamma$  and  $\beta$  using COBYLA by minimising
      $E(\gamma, \beta) = \langle \psi(\gamma, \beta) | H_C | \psi(\gamma, \beta) \rangle$ ;
21   Access the IBM backend ibm_brisbane;
22   Retrieve hardware-calibrated noise model;
23   Initialise the AerSimulator with this noise model;
24   Go to line 9;
  /* Samples Validation */
25 foreach  $s_\tau \in \text{Samples}$  do
26   Substitute  $s_\tau$  into the original combinatorial model;
27   if all constraints are satisfied then
28     Label  $s_\tau$  as feasible ( $s_\tau^f$ );
29   else
30     Label  $s_\tau$  as infeasible ( $s_\tau^u$ );

```

A detailed description of Algorithm 1 is provided below, where line numbers correspond to those in the algorithm. The input is a combinatorial optimisation problem of the type addressed in this study. In line 1, the optimisation model is encoded as a QUBO formulation using either the slack-variable or the unbalanced-penalisation method, as described

in Sections 3.4.1 and 3.4.2. In line 2, the QUBO is transformed into an equivalent Hamiltonian representation through the standard substitution of binary variables $x_i = (1 - z_i)/2$.

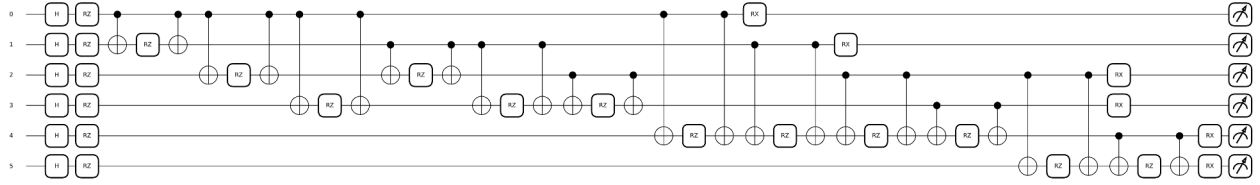
In the first part (lines 1–2), the cost and mixer Hamiltonians are initialised. The cost Hamiltonian H_C corresponds to the Ising model in Eq. 4, where each spin variable z_q is represented by the Pauli-Z operator $z_i = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$. The mixer Hamiltonian H_M facilitates exploration of the solution space and is defined as $H_M = X_1 \otimes X_2 \otimes \dots \otimes X_n$, where $X_q = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ is the Pauli-X (bit-flip) operator acting on qubit q .

In the circuit preparation and measurement (lines 5–23), the number of layers (p) and shots (S) is set (line 5). They significantly affect the performance of the QAOA circuits implemented in both PennyLane and Qiskit. The parameter p controls the circuit depth and the expressibility of the quantum ansatz; increasing it allows the algorithm to explore a richer solution space but also raises computational cost and, in noisy environments, the accumulation of gate errors. In the noiseless PennyLane simulations, larger p values consistently improve approximation quality, while in Qiskit, higher depths on the noisy backend `ibm_brisbane` led to reduced fidelity due to decoherence. The number of shots S defines how many times the circuit is executed to estimate expectation values. In PennyLane, increasing S reduces sampling noise, whereas in Qiskit, it improves measurement accuracy at the expense of longer runtime. Therefore, p and S jointly determine the balance between solution quality, resource usage, and noise resilience.

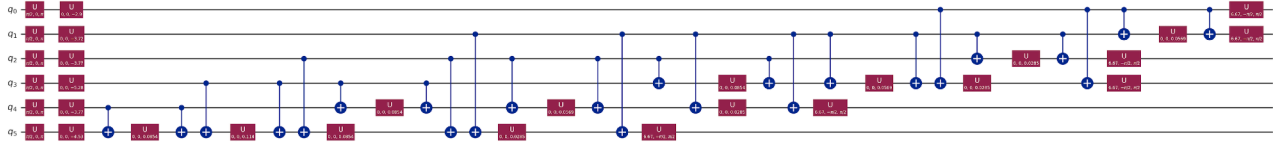
In line 6, a data frame is created to store the samples returned at the end of the algorithm. In the case of PennyLane (lines 7–16), the variational parameters are initialised as linearly spaced values between 0 and 1. The vector γ increases from 0 to 1 across the QAOA layers, while β decreases from 1 to 0, providing balanced initial conditions for the optimisation. For each shot τ (line 9), the quantum system is initialised to $|\psi_0\rangle$ by applying a Hadamard gate $H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$ to every qubit, starting from the all-zero state $|0\rangle^{\otimes n}$ (line 10). In each layer $l = 1, \dots, p$, the cost and mixer operators (Eq. 9) are applied using parameters γ_l and β_l (lines 11–12). After the final layer ($l = p$) is executed, the resulting quantum state is obtained as in Eq. 10 (line 13), and the probability of each state in every shot is computed using Eq. 11 (line 14). Finally, the sample s_τ is stored in the data frame `Samples`, and the validation phase begins in line 24.

In the Qiskit branch (lines 17–23), the algorithm executes the QAOA circuit under realistic noisy conditions with parameter optimisation. In line 18, a parameterised quantum circuit, referred to as the ansatz, is created from the Ising Hamiltonian, incorporating alternating cost and mixer unitaries as defined in Eq. 9. In line 19, the Constrained Optimisation BY Linear Approximations (COBYLA) optimiser adjusts the variational parameters γ and β by minimising the expected energy $E(\gamma, \beta)$, providing a robust derivative-free optimisation method suitable for noisy simulations. COBYLA operates by iteratively approximating the feasible region with a set of linear constraints and updating the solution within a trust region, without requiring gradient information. This makes it robust to stochastic fluctuations arising from quantum measurements and hardware noise.

In line 20, the IBM hardware-calibrated backend `ibm_brisbane` is accessed through the `QiskitRuntimeService` to obtain current device calibration data, including qubit connectivity, gate error rates, and readout error probabilities. In line 21, these parameters are used to build a hardware-aligned noise model, which is then employed in line 22 to initialise the `AerSimulator`. This simulator reproduces the behaviour of quantum hardware by incorporating device-specific features such as gate and readout errors, decoherence times, and coupling topology. In this work, the statevector simulation method was applied to balance computational precision and efficiency. Finally, line 23 redirects execution to line 9, where the optimised and noise-modelled circuit is executed repeatedly for S shots, enabling statistical sampling of noisy QAOA outcomes.



PennyLane: Example of the decomposed QAOA circuit for the FL6 instance (Fig. 2a) with depth 1. The circuit shows the initial Hadamard layer, the cost Hamiltonian encoded through R_Z rotations and controlled-phase interactions, followed by the mixer layer of R_X rotations and final measurements.



Qiskit: Example of the decomposed QAOA circuit for the FL6 instance (Fig. 2a) with depth 1. Each red square labelled $U(\theta, \phi, \lambda)$ denotes a single-qubit rotation in Qiskit form: $U(\frac{\pi}{2}, 0, \pi) = H$, $U(0, 0, \gamma) = R_Z(\gamma)$, and $U(\beta, -\frac{\pi}{2}, \frac{\pi}{2}) = R_X(\beta)$. Two-qubit controlled operations represent the ZZ interactions from the QUBO Hamiltonian, followed by the mixer layer and final measurements.

Fig. 1. Comparison of the fully decomposed QAOA circuit for the FL6 instance produced by PennyLane (left) and Qiskit (right).

In the third and last part of the [Algorithm 1](#) (lines 24 - 29) performs the validation of all bitstring samples collected during the quantum executions. In line 24, the algorithm iterates over each sample $s_r \in \text{Samples}$ obtained from either PennyLane or Qiskit. In line 25, every bitstring $s_r = (s_1, \dots, s_n)$ is substituted into the original combinatorial model to evaluate constraint satisfaction. The conditional check in line 26 verifies whether all model constraints are fulfilled. If this condition holds, the sample is labelled as feasible, denoted s_r^f (line 27). Otherwise, it is classified as infeasible, denoted s_r^u (lines 28-29). This post-processing step allows distinguishing valid from invalid configurations, ensuring that only feasible solutions are considered when analysing the QAOA performance. The ratio of feasible samples provides an empirical measure of the algorithm's ability to generate constraint-satisfying solutions under both noiseless and noisy conditions.

[Fig. 1](#) compares the fully decomposed QAOA circuits produced by PennyLane and Qiskit for the FL6 instance at depth $p = 1$. The PennyLane circuit begins with a layer of Hadamard gates that prepares all qubits in a uniform superposition. The cost Hamiltonian is then implemented through explicit $R_Z(\gamma_i)$ rotations for the linear Z_i terms and CNOT - R_Z - CNOT patterns for the quadratic $Z_i Z_j$ interactions. PennyLane presents each part of the QAOA layer clearly, keeping the structure of the cost and mixer Hamiltonians visible. The mixer Hamiltonian is applied through single qubit $R_X(\beta_i)$ rotations on all qubits, followed by measurement in the computational basis.

The Qiskit circuit applies the same QAOA transformation but uses the IBM native single qubit gate $U(\theta, \phi, \lambda)$. In this format the Hadamard gate is written as $U(\pi/2, 0, \pi)$, the cost rotation as $U(0, 0, \gamma)$, and the mixer rotation as $U(\beta, -\pi/2, \pi/2)$. As in PennyLane, the quadratic interactions are produced through CNOT - rotation - CNOT constructions, although Qiskit may reorder or merge gates depending on the optimisation level used.

Although the two circuits differ visually, both implement the same QAOA layer. PennyLane displays abstract gates such as H , R_Z , and R_X directly, while Qiskit presents the equivalent hardware level U gates used by IBM backends. In both cases the cost Hamiltonian, the mixer Hamiltonian, and the measurement stage are encoded in the same way, with differences arising only from the chosen gate decomposition.

Circuit depth in IBM's quantum devices refers to the number of sequential gate layers that must be executed on the hardware. In practice,

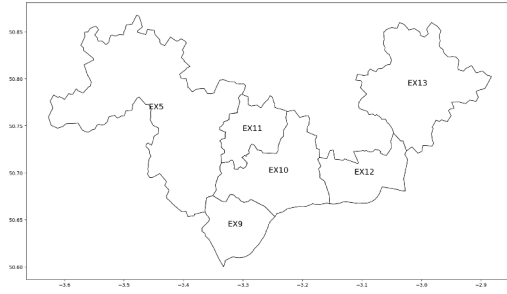
it represents the longest path from the beginning to the end of the circuit when gates that act on different qubits can be executed in parallel. A larger circuit depth implies a longer execution time and greater exposure to decoherence, gate noise, and readout errors, all of which accumulate across the layers of the computation.

On IBM backends, circuit depth is particularly important because two-qubit gates—which dominate QAOA cost Hamiltonians—have significantly higher error rates and longer durations than single-qubit rotations. As depth increases, the probability that the computation completes without accumulating disruptive noise decreases substantially. Therefore, circuits with shallow depth are generally more robust and more likely to return feasible or optimal solutions, whereas deeper circuits suffer from performance degradation due to hardware noise and limited coherence times.

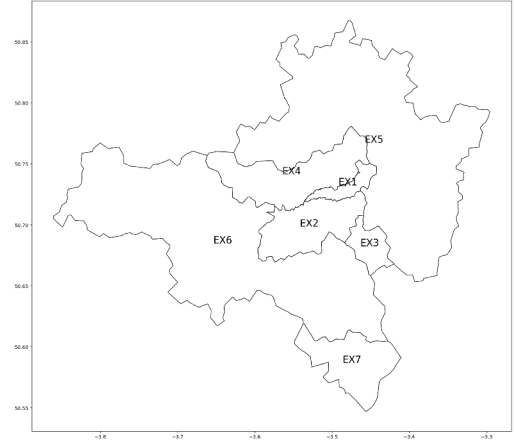
5. Results

To assess the performance of the proposed [Algorithm 1](#), three instances of the facility location (FL) problem and three instances of the line balancing (LB) problem were selected and analysed, as presented in [Fig. 2](#). The FL instances ([Figs. 2a, 2b, and 2c](#)) correspond to real post-code regions located in the south west of England. These areas represent geographically distributed demand zones that must be served within time-critical response windows, thereby providing realistic and operationally relevant test cases for evaluating the algorithm's capability to determine optimal facility placement. In contrast, the three LB instances are benchmark examples commonly reported in the literature, characterised by precedence relations that are challenging to satisfy even for a relatively small number of tasks, thus offering stringent conditions for evaluating the algorithm's performance.

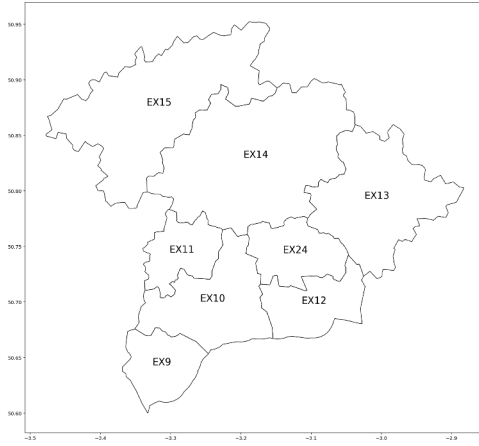
[Table a](#) reports the qubit requirements for each instance under the slack-variable method (VSM) and the unbalanced penalisation method (UPM). For all FL and LB instances, VSM introduces a large overhead due to the additional binary slack variables, quickly surpassing the feasible qubit capacity of the available hardware. In contrast, UPM uses only the original decision variables, yielding a compact encoding that keeps all instances computationally tractable on current quantum devices. This highlights the critical role of the chosen QUBO formulation in enabling practical quantum optimisation.



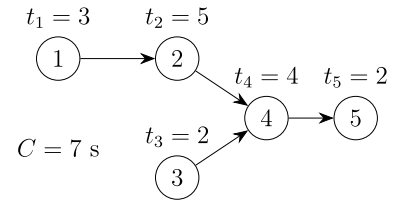
(a) Facility Location Instance FL6



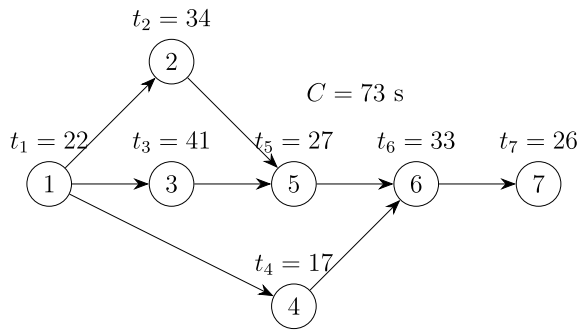
(b) Facility Location Instance FL7



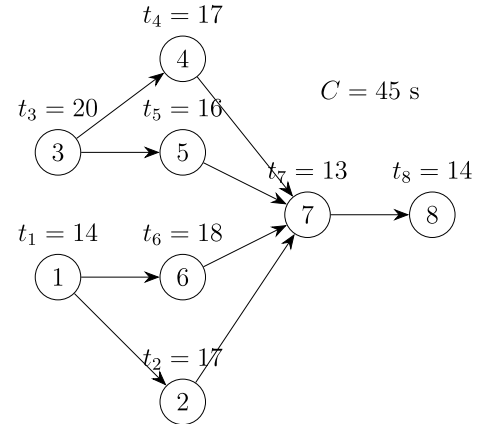
(c) Facility Location Instance FL8



(d) Load Balancing Instance LB5



(e) Load Balancing Instance LB7



(f) Load Balancing Instance LB8

Fig. 2. Instances of the Facility Location (FL) and Load Balancing (LB) Problems.

The computational experiments were conducted on an HP ZBook Studio G10 mobile workstation equipped with an Intel Core i7-13700H processor (13th generation), 32 GB of DDR-5600 memory, and an NVIDIA RTX 4070 GPU with 8 GB of dedicated GDDR6 memory.

5.1. Performance under ideal (noiseless) pennylane conditions

5.1.1. Facility location problem under noiseless conditions

The results of Algorithm 1 when PennyLane is used are presented in Table 2, Fig. 3, and Fig. 4. Table 2 shows the results of setting the values

of number of layers p , shots S , and λ for the VSM or λ_1 and λ_2 when UPM is used to encode the FL problem (see §3.5). For each configuration of algorithmic parameters, the table reports the mean and standard deviation of two performance indicators-the proportion of feasible solutions and the proportion of optimal solutions-computed over repeated runs. The CPU time per experimental configuration is included to quantify the computational effort associated with each QUBO variant.

The runs 1 -15 in Table 2 are run using the parameter $[p, S, \lambda_1, \lambda_2]$ in Eq. 6. The parameter p denotes the number of QAOA layers and determines the linearly spaced angle schedules $\gamma = [0, 1/p, \dots, p/p]$ and

Table 1
Qubit counts by instance.

Instance	Binary variables	Number of constraints	Number of qubits (n)	
			VSM	UPM
FL6	6	6	18	6
FL7	7	7	24	7
FL8	8	8	25	8
LB5	15	12	32 ^{†a}	15
LB6	21	18	40 ^{†a}	21
LB7	24	20	45 ^{†a}	24

^a [†] Not solvable within the capabilities of the available hardware.

Table 2
Results of Algorithm 1 for the FL instances using PennyLane.

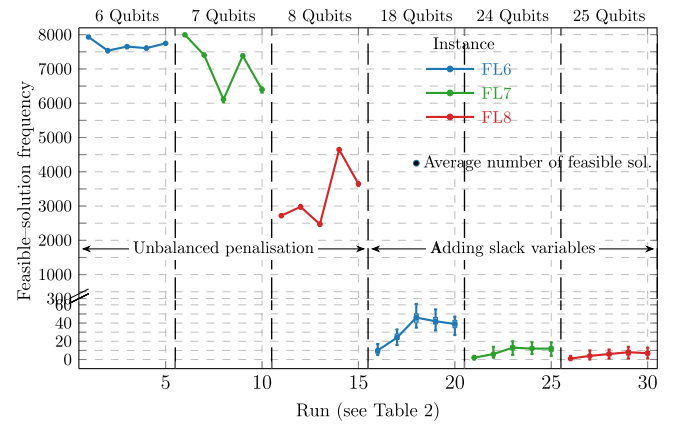
Run	Instance	Parameters $\ddagger, \S, \S, \S$	% Feasible sol.		% Optimal sol.		CPU Time (s)
			Mean	Std. Dev.	Mean	Std. Dev.	
1	FL6 ^{‡a}	[30, 8000, 0.5, 0.01]	99.90	0.03	24.49	0.57	0.11
2		[30, 8000, 1.0, 0.5]	89.51	0.37	87.55	0.36	0.10
3		[30, 8000, 1.0, 0.9]	61.57	0.58	97.81	0.19	0.12
4		[30, 8000, 1.5, 1.0]	80.63	0.52	63.46	0.64	0.11
5		[30, 8000, 1.5, 1.4]	60.85	0.50	84.15	0.48	0.11
6	FL7 ^{‡a}	[30, 8000, 0.5, 0.01]	99.94	0.03	0.62	0.08	0.15
7		[30, 8000, 1.0, 0.5]	92.54	0.26	45.79	0.60	0.15
8		[30, 8000, 1.0, 0.9]	76.39	0.39	60.29	0.66	0.16
9		[30, 8000, 1.5, 1.0]	92.28	0.24	40.34	0.57	0.14
10		[30, 8000, 1.5, 1.4]	79.94	0.40	54.40	0.74	0.15
11	FL8 ^{‡a}	[20, 3000, 1, 0.5]	90.60	0.60	61.85	0.92	0.09
12		[20, 5000, 2.0, 1.5]	59.60	0.60	25.07	0.80	0.09
13		[30, 3000, 1.5, 1.0]	82.30	0.80	44.34	1.13	0.13
14		[30, 5000, 1.0, 0.5]	92.90	0.40	82.64	0.62	0.14
15		[30, 5000, 2.0, 1.5]	72.90	0.50	32.69	0.76	0.14
16	FL6 ^{‡b}	[10, 3000, 1]	10.95	0.62	43.14	3.39	0.34
17		[20, 3000, 5.0]	20.89	0.71	22.05	2.23	0.60
18		[20, 5000, 0.5]	11.05	0.49	97.66	0.73	0.63
19		[20, 5000, 1.0]	24.50	0.61	68.49	1.12	0.64
20		[20, 5000, 10.0]	20.23	0.70	18.19	1.01	0.63
21	FL7 ^{‡b}	[10, 3000, 1]	0.05	0.03	42.28	39.01	15.33
22		[20, 3000, 5.0]	0.20	0.10	30.54	21.85	30.51
23		[20, 5000, 0.5]	0.27	0.07	29.34	11.99	30.54
24		[20, 5000, 1.0]	0.25	0.06	16.87	11.94	30.53
25		[20, 5000, 10.0]	0.23	0.08	7.33	8.85	30.52
26	FL8 ^{‡b}	[10, 3000, 1]	0.04	0.04	4.55	21.32	31.98
27		[20, 3000, 5.0]	0.14	0.07	38.30	21.24	63.77
28		[20, 5000, 0.5]	0.13	0.05	39.47	21.57	64.17
29		[20, 5000, 1.0]	0.15	0.06	44.28	19.92	64.48
30		[20, 5000, 10.0]	0.14	0.06	0.42	2.28	64.03

^a [‡] Parameters $[p, S, \lambda_1, \lambda_2]$ for the unbalanced penalisation method, see Eq. 6.

^b [‡] Parameters $[p, S, \lambda]$ for the slack-variable method, see Eq. 5.

$\beta = [p/p, \dots, 1/p, 0]$. The penalty parameters λ_1 and λ_2 follow the recommendations in [11,40]. The percentage of feasible solutions is computed as the proportion of feasible outcomes with respect to the total number of shots p , while the percentage of optimal solutions corresponds to the proportion of optimal outcomes among all feasible ones. For FL6 and FL7, the configuration [30, 8000, 0.5, 0.01] yields the highest feasibility rates, although not the highest proportion of optimal solutions. For FL8, both the largest feasibility and optimality values are achieved with the configuration [30, 5000, 1, 0.5].

Runs 16–30 in Table 2 use the parameter set $[p, S, \lambda]$ corresponding to the slack-variable formulation in Eq. 5, with γ and β defined as before. In this setting, the configuration [20, 5000, 1] yields the highest feasibility and optimality across the three instances. However, the overall feasibility achieved with slack variables is substantially lower than with the unbalanced penalisation approach. Even in the best case

**Fig. 3.** Feasible-solution frequency for Facility Location instances FL6, FL7, and FL8.**Table 3**
Results of Algorithm 1 to solve the LB instances using the UPM in PennyLane.

Run	Instance	Parameters $[p, S, \lambda_1, \lambda_2]$	% Optimal solutions		CPU Time (s)
			Mean	Std. dev.	
1	LB5	[15, 5000, 0.1, 0.01]	1.36	0.16	0.28
2		[20, 2000, 0.5, 0.025]	4.01	0.47	0.36
3		[20, 3000, 0.5, 0.025]	4.00	0.42	0.36
4		[20, 5000, 0.3, 0.01]	3.12	0.29	0.35
5		[20, 5000, 0.5, 0.025]	3.99	0.26	0.34
6	LB7	[15, 6000, 0.008, 0.0008]	0.06	0.03	13.49
7		[20, 5000, 0.001, 0.0001]	0.06	0.04	17.99
8		[20, 5000, 0.005, 0.0005]	0.07	0.04	18.03
9		[20, 5000, 0.008, 0.0008]	0.08	0.03	18.02
10		[20, 6000, 0.005, 0.0005]	0.07	0.03	17.94
11	LB8	[15, 3000, 0.001, 0.0001]	0.09	0.06	133.52
12		[15, 5000, 0.001, 0.0001]	0.09	0.04	129.89
13		[15, 5000, 0.005, 0.0005]	0.22	0.06	132.85
14		[15, 5000, 0.009, 0.0009]	0.25	0.07	132.42
15		[15, 5000, 0.01, 0.001]	0.27	0.06	131.19

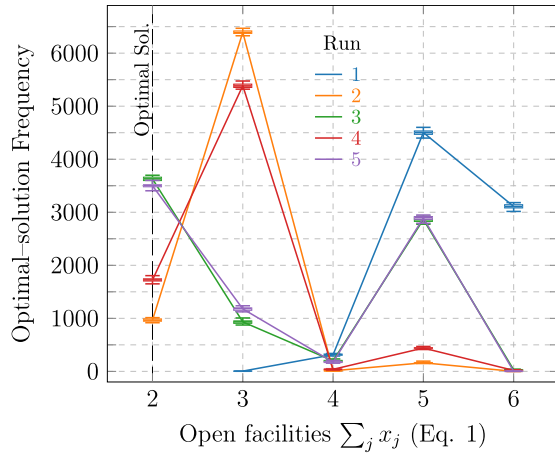
(run 19), the proportion of feasible solutions reaches only 25.5 % for the smallest instance (LB5).

Fig. 3 compares the number of feasible solutions obtained by QAOA the runs in Table 2. Under unbalanced penalisation, all instances yield high feasible-solution frequencies: FL6 and FL7 consistently return close to 8,000 feasible solutions, while FL8 achieves between 2,500 and 4,500. In contrast, the slack-variable formulation exhibits a pronounced decline. Due to the substantially larger qubit requirements (18–25 qubits), feasible solutions become scarce, with most runs producing fewer than 300 feasible outcomes. Overall, the results show that unbalanced penalisation provides far more reliable and scalable QAOA performance, whereas slack variables severely limit the algorithm's ability to reach the feasible region.

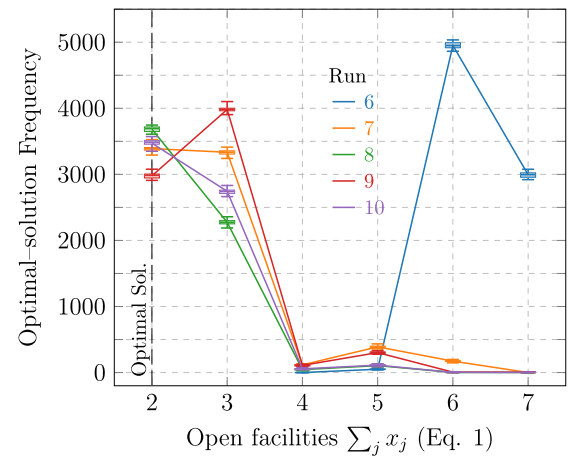
The Fig. 4 reports the frequency with which QAOA identifies the optimal solution for the three FL instances under the unbalanced penalisation method (UPM). In all cases, QAOA concentrates probability around the true optimal configuration (marked by the dashed line), with clearly dominant peaks compared with neighbouring values. Although the magnitude of these frequencies varies across runs and instances, the optimal configuration is consistently the most likely, demonstrating that the unbalanced penalisation method reliably steers QAOA towards the correct region of the solution space.

5.1.2. Load balancing problem under noiseless conditions

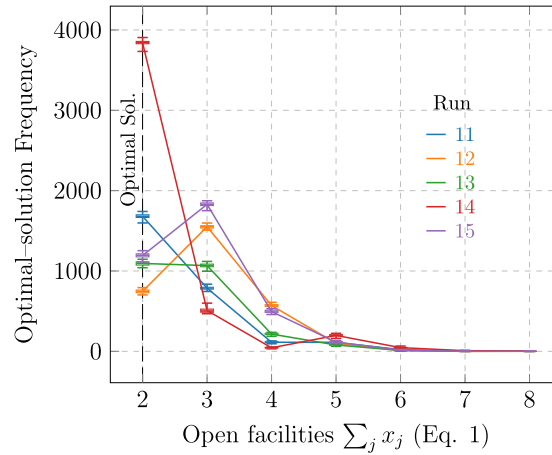
As shown in Table 3, the LB5 results indicate that runs 2, 3, and 5—all using $p = 20$, $\lambda_1 = 0.5$, and $\lambda_2 = 0.025$ —achieve the highest optimality rates, reaching up to 4.01 % with low variability. This suggests



(a) IFL6-Optimal-solution frequency
(6 qubits)



(b) FL7-Optimal-solution frequency
(7 qubits)



(c) FL8-Optimal-solution frequency
(8 qubits)

Fig. 4. Optimal-solution frequency for Facility Location instances FL6, FL7, and FL8 using UPM, see Table 2.

that moderate penalisation strengths and a circuit depth of $p = 20$ are well suited to the relatively simple precedence structure of LB5 (Fig. 2d).

For LB7, all configurations yield substantially lower optimality, with a maximum of only 0.08 % (run 9). The more complex graph structure and longer task chains (Fig. 2e) make the instance considerably harder to satisfy, indicating higher sensitivity to parameter choices.

LB8 also shows low optimality but exhibits a clearer improvement with increasing penalisation. The best performance, 0.27 %, occurs in run 15 with $\lambda_1 = 0.01$ and $\lambda_2 = 0.001$, suggesting that stronger constraint enforcement helps mitigate the instance's structural complexity.

Fig. 5 further illustrates the frequency with which QAOA finds the optimal solution. LB5 displays a steady increase across runs, exceeding 200 optimal solutions in the best case. LB7 remains consistently low across all settings, while LB8 shows a gradual rise from runs 11 to 15, peaking at 14 optimal solutions with parameters [15, 5000, 0.01, 0.001]. Overall, LB5 is significantly more amenable to QAOA than LB7 and LB8, whose complex precedence constraints lead to markedly lower optimality frequencies.

The results in Table 2 and Table 3 show that FL instances benefit greatly from the unbalanced penalisation method, achieving high feasibility and optimality across several parameter settings. In contrast, LB instances exhibit much lower optimality, especially for the more complex LB7 and LB8 cases, though LB5 performs reasonably well. These

outcomes emphasise the strong effect of problem structure and the need for careful parameter selection in QAOA.

5.2. Performance under noisy and optimisation conditions

5.2.1. Facility location problem under noisy conditions

The performance of the proposed quantum-inspired solution method (Algorithm 1) was further examined under realistic noisy conditions, using hardware-calibrated noise models derived from IBM quantum devices. This analysis evaluates the robustness of the unbalanced penalisation formulation when subject to decoherence, gate errors, and readout noise—factors that strongly influence the quality of QAOA outputs on current NISQ hardware.

The boxplots in Fig. 6 compare the performance of the unbalanced penalisation (left groups) and slack-variable (right groups) formulations for the FL6–FL8 instances under noisy Qiskit simulations with parameter optimisation. For all instances, the unbalanced penalisation method yields substantially higher numbers of feasible solutions, with distributions concentrated well above those obtained using slack variables. This behaviour becomes more pronounced as instance size increases: while FL6 and FL7 still produce a moderate number of feasible outcomes under the slack-based encoding, FL8 exhibits a marked degradation, reflecting the increasing difficulty of maintaining feasibility when

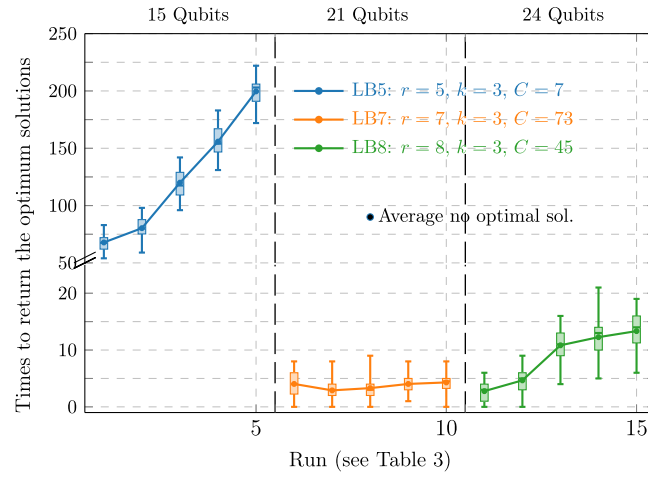


Fig. 5. Optimal-solution frequency for Load Balancing instances LB5, LB7, and LB8. (r = tasks, k = stations, and C = cycle time).

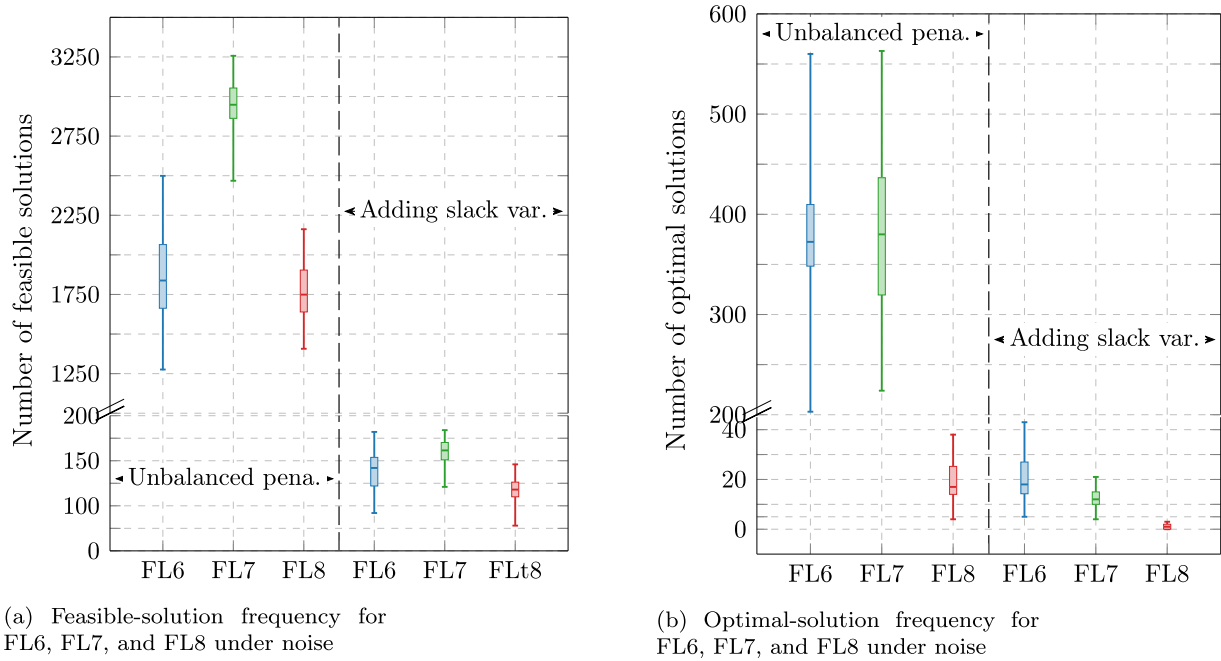


Fig. 6. Feasible- and optimal-solution frequencies for Facility Location instances FL6, FL7, and FL8 under noisy Qiskit simulations.

Table 4

Results of the Algorithm 1 for the instances Fig. 2a, 2b and 2c.

Instance	Encoding method	Parameters [†]	Circuit depth	CPU time (s)	γ and β optimisation	IBM Quantum backend
FL6	UPM ^a	[30,5000,0.01,0.5]	1460	1.14	COBYLA	ibm_brisbane
FL7		[30,5000,0.01,0.5]	1736	2.00		
FL8		[30,5000,0.01,0.5]	1990	1.90		
FL6	SVM ^b	[3,250,0.5]	443	23.1		
FL7		[3,250,1.0]	602	5808.2		
FL8		[3,250,1.0]	628	5511.4		

^a $\dagger [p, S, \lambda_1, \lambda_2]$ for the unbalanced penalisation method (UPM).

^b $\dagger [p, S, \lambda]$ for the slack variable method (SVM).

additional slack qubits amplify noise exposure. A similar pattern is observed for the optimal-solution frequencies. The unbalanced penalisation consistently achieves broader, higher-valued distributions, whereas the slack-variable method leads to sharply reduced optimal counts, often approaching zero for the largest instance. Overall, the figure highlights the robustness of the unbalanced penalisation formulation under realis-

tic noisy conditions, underscoring its suitability for near-term quantum hardware.

Table 4 summarises the behaviour of QAOA under two encoding strategies—UPM and SVM—when executed on the *ibm_brisbane* backend with parameters optimised using COBYLA. Three key trends can be observed.

Table 5
Results of the Algorithm 1 for the instances Fig. 2a, 2b, and 2c.

Instance	Encoding method	Parameters $[p, S, \lambda_1, \lambda_2]$	Circuit depth	CPU time (s)	γ and β optimisation	IBM Quantum backend
LB5	UPM	[10,8000,1,12]	19,725	24.6	COBYLA	ibm_brisbane
LB7		[5,8000,0.005,0.0005]	18,673	190.1		
LB8		[5,8000,1,12]	26,360	2541.0		

First, for all FL instances, the UPM formulation produces significantly deeper circuits (1,460–1,990 layers) than SVM (443–628 layers). This is expected, as UPM embeds additional penalty terms directly in the cost Hamiltonian. Despite the greater circuit depth, UPM achieves markedly lower CPU times (approximately 1–2 s), indicating that circuit depth alone does not determine the computational cost on IBM hardware. Instead, UPM benefits from a more stable parameter landscape and smoother optimisation behaviour.

Second, the SVM formulation yields much shorter circuits but exhibits extremely high CPU times, particularly for FL7 and FL8 (around 5,500–5,800 s). These results reflect the optimisation difficulties inherent in slack-variable encodings: the resulting landscapes are steeper and more irregular, requiring substantially more optimisation iterations, which dominate the total runtime. This behaviour is consistent with the low feasibility and optimality rates previously reported for SVM.

Third, runtime variability across instances remains small under UPM but grows substantially under SVM, indicating that UPM provides a more scalable and predictable encoding for FL problems on NISQ devices. Overall, the table highlights that slack-variable approaches are not suitable for QAOA on current hardware, both in terms of computational cost and solution quality.

5.2.2. Load balancing problem under noisy conditions

Table 5 and Fig. 7 present the QAOA performance for the load balancing instances when the unbalanced penalisation method is executed on the `ibm_brisbane` backend. The results reveal a clear dependence on circuit depth and problem size. For LB5, the smallest instance, the circuit depth is already substantial (19,725 layers), yet QAOA manages to recover the optimal solution up to three times across the evaluated shots. This indicates that, despite noise, the algorithm retains some capacity to navigate the search space when the instance is relatively compact. However, as the problem size increases, the feasibility of obtaining optimal solutions deteriorates sharply. LB7 and LB8 require circuit depths of 18,673 and 26,360, respectively, resulting in a dramatic increase in execution times and near-zero optimal-solution recovery. In both cases, QAOA fails to identify the optimal solution in almost all repetitions, demonstrating that the accumulated noise across thousands of layers overwhelms the variational circuit and prevents meaningful optimisation.

Overall, the results underscore the sensitivity of QAOA to circuit depth under realistic hardware noise. While unbalanced penalisation remains theoretically attractive, its practical use for load balancing on current devices is limited to very small instances. Larger problems demand circuit depths far beyond the noise tolerance of today's NISQ hardware, making improved encoding strategies, depth-reduction techniques, or error-mitigation methods essential for achieving reliable performance on industrial-scale load balancing formulations.

Across all experiments, a clear contrast emerges between the noiseless and noisy settings. Under ideal (noiseless) conditions, the unbalanced penalisation method enables QAOA to achieve high feasibility and, in many cases, strong optimality, particularly for the FL instances and the smaller LB5 case. However, when realistic hardware noise is introduced through IBM-calibrated models, performance deteriorates noticeably. The degradation is far more severe for LB instances: despite acceptable behaviour in noiseless simulations, noisy executions return few or no optimal solutions for LB7 and LB8 due to the prohibitive circuit depths required. Overall, the comparison shows that while QAOA can

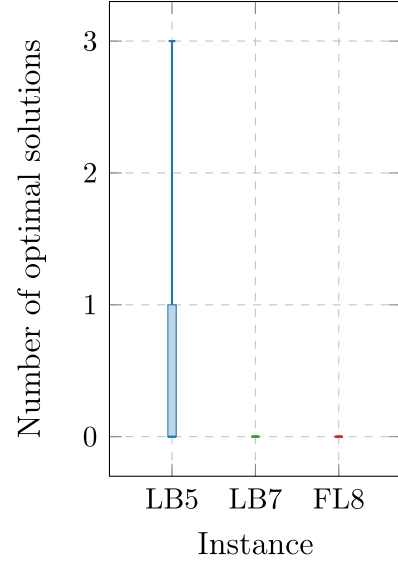


Fig. 7. Optimal-solution frequency for Load Balancing instances LB5, LB7, and LB8 under noisy Qiskit simulations.

perform well in noiseless environments, hardware noise imposes strict practical limits, especially for deeper circuits and structurally complex problems.

6. Discussion

This study explored the applicability of the Quantum Approximate Optimisation Algorithm (QAOA) to two fundamental combinatorial optimisation problems in supply chain management: facility location (FL) and load balancing (LB). Both problems were reformulated as QUBO models and implemented using two encoding strategies—slack variables and unbalanced penalisation. The results demonstrate that QAOA can successfully identify optimal solutions for small- to medium-scale problem instances, depending on parameter selection, encoding choice, and the presence or absence of hardware noise.

As shown in Table 2, when the FL problem is encoded using the unbalanced penalisation method, QAOA identifies the optimal solution more frequently across all three evaluated instances (Fig. 2a, Fig. 2b, and Fig. 2c). These results were obtained using penalisation parameters in the range of 0.5 to 1.0, as recommended in [11, 40]. A similar pattern is observed in the LB problem (Table 3), where the unbalanced penalisation method outperforms the slack-variable encoding in terms of optimality rate. This behaviour is consistent with the exponential growth of the solution space as the number of qubits increases, which reduces QAOA's ability to amplify the probability of the optimal state [41].

An important contribution of this study is the inclusion of noisy QAOA experiments using Qiskit and a hardware-calibrated noise model for the `ibm_brisbane` backend. In these experiments, the variational parameters were optimised using the derivative-free COBYLA algorithm, which iteratively minimises the expected energy of the cost Hamiltonian. The noisy results reveal a clear degradation in performance compared with noiseless PennyLane simulations. For FL instances, the un-

balanced penalisation method still recovers feasible and optimal solutions under noise, demonstrating some robustness to decoherence and gate errors. In contrast, LB instances suffer substantial performance loss: due to their denser constraint structure and deeper circuit requirements, noisy QAOA finds very few or no optimal solutions for LB7 and LB8 (as expected according to [42]). These outcomes reflect the cumulative effect of noise in long-depth circuits and highlight the need for hardware-aware optimisation strategies.

The scalability of the proposed QAOA–QUBO framework was evaluated by examining how qubit requirements, circuit depth, and solution quality evolve as instance size increases. The results show that qubit growth is manageable only under the unbalanced penalisation method, which preserves a linear relationship between instance size and number of qubits; in contrast, the slack-variable encoding expands rapidly and becomes infeasible beyond 25–30 qubits. Circuit depth increases sharply with problem size for both FL and LB, but the effect is far more pronounced for LB, where depths exceeding 20,000 layers lead to substantial noise accumulation. This directly affects solution quality: while FL instances maintain high feasibility and frequent optimality in both noiseless and noisy settings, LB instances deteriorate rapidly, with optimal solutions disappearing entirely for LB7 and LB8 under realistic noise. These findings indicate that QAOA remains viable for small and moderately structured supply-chain problems, but its practical scalability is currently constrained by qubit limits, depth-induced noise, and the structural density of the underlying constraints.

The slack-variable method, while intuitive, poses additional scalability challenges. Due to hardware limitations, only instances requiring up to 25 qubits could be processed in this study (see Fig. 5 and Fig. 7). Larger instances—particularly those for the LB problem—become intractable when encoded with slack variables, as the method introduces many additional binary variables, inflating qubit counts and circuit depth (Fig. 2d, Fig. 2e, and Fig. 2f). The noisy QAOA results further emphasise this limitation: slack-variable circuits show unstable COBYLA convergence and often yield negligible feasible solutions under realistic noise.

These findings align with earlier studies employing quantum annealers for related supply chain problems. Logistics networks have been designed using a hybrid classical-quantum annealing scheme on D-Wave, relying on large penalty-based QUBOs, minor embedding, and classical decomposition, without variational circuits, noise analysis, or parameter optimisation [30]. A small-maximal covering problem is solved using D-Wave and a noiseless QAOA simulator, without examining circuit depth, noise effects, or scalability [31]. In contrast, our study develops a full gate-based QAOA framework for two supply-chain problems (facility location and load balancing), compares two QUBO encodings, incorporates hardware-calibrated noise through Qiskit, optimises parameters with COBYLA, and provides a systematic scalability assessment in terms of qubits, circuit depth, and solution quality. Moreover, unlike existing QUBO modelling guides that remain largely theoretical [17–19], our study bridges theory and practice, enabling hands-on experimentation with variational quantum workflows.

Nevertheless, several limitations must be acknowledged. First, the scalability of the method is constrained by the current limits of quantum hardware, as both qubit counts and circuit depth grow rapidly with instance size. Second, QAOA performance is sensitive to hyperparameter selection; in this study, penalisation parameters and QAOA depths were selected based on prior work rather than through automated tuning. Third, the noisy experiments show that parameter optimisation via COBYLA becomes increasingly difficult as circuit depth increases, limiting practical performance on more complex instances.

Future research could explore hybrid quantum–classical algorithms that integrate QAOA with classical metaheuristics, depth-reduction strategies to improve noise resilience, and problem reformulations that reduce the number of binary variables. Automated methods for tuning variational parameters—such as Bayesian optimisation, reinforcement learning, or warm-start strategies—may also improve convergence un-

der noise. As quantum hardware evolves toward higher qubit counts and lower error rates, the proposed framework may become suitable for addressing larger and more realistic supply-chain optimisation tasks.

7. Conclusions

This study explored the application of the Quantum Approximate Optimisation Algorithm (QAOA) to two key supply chain optimisation problems: Facility Location (FL) and Load Balancing (LB). Both were reformulated as QUBO models using slack-variable and unbalanced penalisation encodings and solved through QAOA using PennyLane and hardware-calibrated Qiskit simulations. Across all experiments, the unbalanced penalisation method consistently outperformed the slack-based approach by achieving higher feasibility and optimality rates while requiring significantly fewer qubits. For FL instances, QAOA demonstrated stable and robust performance under moderate penalty parameters, reliably recovering the optimal configuration. LB instances were more challenging due to their denser constraint structure, yet QAOA still identified optimal solutions when parameters were carefully tuned.

This work contributes reproducible, end-to-end implementations that connect theoretical QUBO modelling with practical quantum algorithm deployment for supply chain optimisation. The findings reveal both the promise and the current limitations of QAOA: while small and moderately structured FL and LB instances are tractable, scalability is constrained by circuit depth, noise accumulation, and qubit growth. Future research should prioritise automated parameter-selection strategies, hybrid quantum-classical methods, and problem reformulations that further reduce qubit requirements and improve noise resilience, enabling QAOA to scale toward more realistic industrial applications.

CRedit authorship contribution statement

Luis A. Moncayo-Martínez: Writing – original draft, Validation, Methodology, Formal analysis, Data curation, Conceptualization; **Nai-hui He:** Writing – review & editing, Validation, Methodology, Formal analysis, Conceptualization.

Data availability

GitHub link provided in the manuscript

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

The reviewers for their constructive comments and insights, which greatly improved the quality of this paper. The Leverhulme Trust for supporting this work through the Leverhulme Visiting Professorship (VP2-2023-021).

Financial support from Asociación Mexicana de Cultura, A.C. is gratefully acknowledged. National Council of Humanities, Sciences, and Technologies (CONAHCYT) from Mexico.

References

- [1] C. Gomes, J.P. Fernandes, G. Falcao, S. Kar, S. Tayur, A systematic mapping study on quantum and quantum-inspired algorithms in operations research, *ACM Comput. Surv.* 57 (3) (2024). <https://doi.org/10.1145/3700874>
- [2] K. Bharti, A. Cervera-Lierta, T.H. Kyaw, T. Haug, S. Alperin-Lea, A. Anand, M. Degroote, H. Heimonen, J.S. Kottmann, T. Menke, W.-K. Mok, S. Sim, L.-C. Kwek, A. Aspuru-Guzik, Noisy intermediate-scale quantum algorithms, *Rev. Mod. Phys.* 94 (1) (2022) 015004. <https://doi.org/10.1103/RevModPhys.94.015004>

- [3] M. Núñez-Merino, J.M. Maqueira-Marín, J. Moyano-Fuentes, C.A. Castaño-Moraga, Quantum-Inspired computing technology in operations and logistics management, *Int. J. Phys. Distrib. Logist. Manag.* 54 (3) (2024) 247–274. <https://doi.org/10.1108/IJPDLM-02-2023-0065>
- [4] F. Phillipson, Quantum Computing in Logistics and Supply Chain Management: An Overview, *arXiv preprint arXiv:2402.17520* (2024). <https://arxiv.org/abs/2402.17520>.
- [5] K. Kurowski, T. Pecyna, M. Slys, R. Rafał, G. Waligóra, J. Węclarz, Application of quantum approximate optimization algorithm to job shop scheduling problem, *Eur. J. Oper. Res.* 310 (2) (2023) 518–528. <https://doi.org/10.1016/j.ejor.2023.03.013>
- [6] H. Jing, Y. Wang, Y. Li, Data-driven quantum approximate optimization algorithm for power systems, *Commun. Eng.* 2 (1) (2023) 12. <https://doi.org/10.1038/s44172-023-00061-8>
- [7] M. Vandel, A. Lignarolo, C. Cavazzoni, D. Dragoni, Evaluating the practicality of quantum optimization algorithms for prototypical industrial applications, *Quantum Inf. Process.* 23 (10) (2024) 344. <https://doi.org/10.1007/s11128-024-04560-1>
- [8] S.J. Weinberg, F. Sanches, T. Ide, K. Kamiya, R. Correll, Supply chain logistics with quantum and classical annealing algorithms, *Sci. Rep.* 13 (1) (2023) 4770. <https://doi.org/10.1038/s41598-023-31765-8>
- [9] V. Bergholm, J. Izaac, M. Schuld, C. Gogolin, N. Killoran, PennyLane: Automatic Differentiation of Hybrid Quantum-Classical Computations, *arXiv preprint arXiv:1811.04968* (2020). <https://arxiv.org/abs/1811.04968>.
- [10] A. Javadi-Abhari, M. Treinish, K. Krsulich, C.J. Wood, J. Lishman, J. Gacon, S. Martiel, P.D. Nation, L.S. Bishop, A.W. Cross, B.R. Johnson, J.M. Gambetta, Quantum Computing with Qiskit, 2024, <https://arxiv.org/abs/2405.08810>.
- [11] J.A. Montañez-Barrera, D. Willsch, A. Maldonado-Romo, K. Michielsen, Unbalanced penalization: a new approach to encode inequality constraints of combinatorial problems for quantum optimization algorithms, *Quantum Sci. Technol.* 9 (2) (2024) 025022. <https://doi.org/10.1088/2058-9565/ad35e4>
- [12] P.H. Wang, W.Y. Wu, C.Y. Lee, et al., Advantages of two quantum programming platforms in quantum computing and quantum chemistry, *J. Cheminform.* 17 (77) (2025). <https://doi.org/10.1186/s13321-025-01026-z>
- [13] F. Rodríguez-Díaz, D. Gutiérrez-Avilés, A. Troncoso, F. Martínez-Álvarez, A survey of quantum machine learning: foundations, algorithms, frameworks, data and applications, *ACM Comput. Surv.* 58 (4) (2025) 91:1–91:35. <https://doi.org/10.1145/3764582>
- [14] P.M. Xavier, P. Ripper, T. Andrade, J.D. Garcia, N. Maculan, D.E.B. Neira, Qubo. JI: A Julia Ecosystem for Quadratic Unconstrained Binary Optimization, *arXiv preprint arXiv:2307.02577* (2023).
- [15] U. Tosun, A new tool for automated transformation of quadratic assignment problem instances to quadratic unconstrained binary optimisation models, *Expert Syst. Appl.* 201 (2022) 116953. <https://doi.org/10.1016/j.eswa.2022.116953>
- [16] M. Zaman, K. Tanahashi, S. Tanaka, Pyqubo: python library for mapping combinatorial optimization problems to QUBO form, *IEEE Trans. Comput.* 71 (4) (2021) 838–850. <https://doi.org/10.1109/TC.2021.3063618>
- [17] T. Gabor, M.L. Rosenfeld, C. Linnhoff-Popien, S. Feld, How to approximate any objective function via quadratic unconstrained binary optimization, in: 2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), 2022, pp. 1249–1257. <https://doi.org/10.1109/SANER53432.2022.00149>
- [18] F. Glover, G. Kochenberger, Y. Du, et al., Applications and computational advances for solving the QUBO model, in: A.P. Punnen (Ed.), *The Quadratic Unconstrained Binary Optimization Problem: Theory, Algorithms, and Applications*, Springer International Publishing, Cham, 2022, pp. 39–56. https://doi.org/10.1007/978-3-031-04520-2_2
- [19] F. Glover, G. Kochenberger, R. Hennig, Y. Du, Quantum bridge analytics i: a tutorial on formulating and using QUBO models, *Ann. Oper. Res.* 314 (1) (2022) 141–183. <https://doi.org/10.1007/s10479-022-04634-2>
- [20] M. Cerezo, A. Arrasmith, R. Babbush, S.C. Benjamin, S. Endo, K. Fujii, J.R. McClean, K. Mitarai, X. Yuan, L. Cincio, P.J. Coles, Variational quantum algorithms, *Nat. Rev. Phys.* 3 (9) (2021) 625–644. <https://doi.org/10.1038/s42254-021-00348-9>
- [21] M.P. Harrigan, K.J. Sung, M. Neeley, K.J. Satzinger, F. Arute, K. Arya, J. Atalaya, J.C. Bardin, R. Barends, S. Boixo, M. Broughton, B.B. Buckley, D.A. Buell, B. Burkett, N. Bushnell, Y. Chen, Z. Chen, Ben Chiaro, R. Collins, W. Courtney, S. Demura, A. Dunsworth, D. Eppens, A. Fowler, B. Foxen, C. Gidney, M. Giustina, R. Graff, S. Habegger, A. Ho, S. Hong, T. Huang, L.B. Ioffe, S.V. Isakov, E. Jeffrey, Z. Jiang, C. Jones, D. Kafri, K. Kechedzhi, J. Kelly, S. Kim, P.V. Klimov, A.N. Korotkov, F. Kostritsa, D. Landhuis, P. Laptev, M. Lindmark, M. Leib, O. Martin, J.M. Martinis, J.R. McClean, M. McEwen, A. Megrant, X. Mi, M. Mohseni, W. Mruczkiewicz, J. Mutus, O. Naaman, C. Neill, F. Neukart, M.Y. Niu, T.E. O'Brien, B. O'Gorman, E. Ostby, A. Petukhov, H. Putterman, C. Quintana, P. Roushan, N.C. Rubin, D. Sank, A. Skolik, V. Smelyanskiy, D. Strain, M. Streif, M. Szalay, A. Vainsencher, T. White, Z.J. Yao, P. Yeh, A. Zalcman, L. Zhou, H. Neven, D. Bacon, E. Lucero, E. Farhi, R. Babbush, Quantum approximate optimization of non-Planar graph problems on a planar superconducting processor, *Nat. Phys.* 17 (3) (2021) 332–336. <https://doi.org/10.1038/s41567-020-01105-y>
- [22] S.H. Sack, D.J. Egger, Large-Scale quantum approximate optimization on nonplanar graphs with machine learning noise mitigation, *Phys. Rev. Res.* 6 (1) (2024) 013223. <https://doi.org/10.1103/PhysRevResearch.6.013223>
- [23] K. Blekos, D. Brand, A. Ceschini, C.-H. Chou, R.-H. Li, K. Pandya, A. Summer, A review on quantum approximate optimization algorithm and its variants, *Phys. Rep.* 1068 (2024) 1–66. <https://doi.org/10.1016/j.physrep.2024.03.002>
- [24] C. Grange, M. Poss, E. Bourreau, An introduction to variational quantum algorithms for combinatorial optimization problems, *Ann. Oper. Res.* 343 (2) (2024) 847–884. <https://doi.org/10.1007/s10479-024-06253-5>
- [25] R. Fakhimi, H. Validi, Quantum approximate optimization algorithm (QAOA), in: P.M. Pardalos, O.A. Prokopyev (Eds.), *Encyclopedia of Optimization*, Springer International Publishing, Cham, 2020, pp. 1–7. https://doi.org/10.1007/978-3-030-54621-2_854-1
- [26] E. Farhi, S. Gutmann, D. Ranard, B. Villalonga, Lower Bounding the MaxCut of High Girth 3-Regular Graphs Using the QAOA, *arXiv preprint arXiv:2503.12789* (2025).
- [27] S. Yarkoni, E. Raponi, T. Bäck, S. Schmitt, Quantum annealing for industry applications: introduction and review, *Rep. Prog. Phys.* 85 (10) (2022) 104001. <https://doi.org/10.1088/1361-6633/ac8c54>
- [28] D. Herman, C. Googin, X. Liu, Y. Sun, A. Galda, I. Saffro, M. Pistoia, Y. Alexeev, Quantum computing for finance, *Nat. Rev. Phys.* 5 (8) (2023) 450–465. <https://doi.org/10.1038/s42254-023-00603-1>
- [29] S.-S. Wang, H.-L. Liu, Y.-M. Li, F. Gao, S.-J. Qin, Q.-Y. Wen, Variational quantum algorithm-Preserving feasible space for solving the uncapacitated facility location problem, *Adv. Quantum Technol.* (2024) 2400201. <https://doi.org/10.1002/qute.202400201>
- [30] Y. Ding, X. Chen, L. Lamata, E. Solano, M. Sanz, Implementation of a hybrid classical-Quantum annealing algorithm for logistic network design, *SN Comput. Sci.* 2 (2) (2021) 68. <https://doi.org/10.1007/s42979-021-00466-2>
- [31] A. Giraldo-Quintero, J.G. Lalinde-Pulido, J.C. Duque, D. Sierra-Sosa, Using quantum computing to solve the maximal covering location problem, *Comput. Urban Sci.* 2 (1) (2022) 43. <https://doi.org/10.1007/s43762-022-00070-x>
- [32] O. Battaila, X. Delorme, A. Dolgui, H. Haddou-Benderbal, New trends in line balancing and model sequencing in assembly, disassembly and machining environments, *Comp. Indust. Eng.* (2025) 111210. <https://doi.org/10.1016/j.cie.2025.111210>
- [33] M. Willmann, M. Albus, J. Schnabel, M. Roth, Application of Quantum Annealing for Scalable Robotic Assembly Line Optimization: A Case Study, *arXiv preprint arXiv:2412.09239* (2024).
- [34] R.K. Singh, A.R. Singh, R.K. Yadav, A balanced-Quantum inspired evolutionary algorithm for solving disassembly line balancing problem, *Appl. Soft. Comput.* 132 (2023) 109840. <https://doi.org/10.1016/j.asoc.2022.109840>
- [35] C.-T. Lai, C. Blank, P. Schmelcher, R. Mukherjee, Towards arbitrary QUBO optimization: analysis of classical and quantum-Activated feedforward neural networks, *Mach. Learn.* 6 (2) (2025) 025049. <https://doi.org/10.1088/2632-2153/addb97>
- [36] L.J. Watters, Reduction of integer polynomial programming problems to zero-One linear programming problems, *Oper. Res.* 15 (6) (1967) 1171–1178. <https://doi.org/10.1287/opre.15.6.1171>
- [37] E. Farhi, J. Goldstone, S. Gutmann, A Quantum Approximate Optimization Algorithm, *arXiv preprint arXiv:1411.4028* (2014).
- [38] F. Rodríguez-Díaz, J.F. Torres, D. Gutiérrez-Avilés, A. Troncoso, F. Martínez-Álvarez, An experimental comparison of qiskit and pennylane for hybrid quantum-Classical support vector machines, in: A. Alonso-Betanzos, B. Guijarro-Berdiñas, V. Bolón-Canedo, E. Hernández-Pereira, O. Fontenla-Romero, D. Camacho, J.R. Rabuñal, M. Ojeda-Aciego, J. Medina, J.C. Riquelme, A. Troncoso (Eds.), *Advances in Artificial Intelligence*, Springer Nature Switzerland, Cham, 2024, pp. 121–130. https://doi.org/10.1007/978-3-031-62799-6_13
- [39] M. Hamid, B. Alam, O. Pal, Comparative study of quantum computing tools and frameworks, in: *Innovation and Emerging Trends in Computing and Information Technologies – IETCIT 2024, 2126 of Communications in Computer and Information Science*, Springer, Cham, 2025, pp. 87–104. https://doi.org/10.1007/978-3-031-80842-5_8
- [40] E. Güneş, J. Ehrenthal, T. Hanne, QUBO Formulations and characterization of penalty parameters for the multi-Knapsack problem, *IEEE Access* 13 (2025) 47086–47098. <https://doi.org/10.1109/ACCESS.2025.3550788>
- [41] P.C. Lotshaw, T. Nguyen, A. Santana, A. McCaskey, R. Herrman, J. Ostrowski, G. Siopsis, T.S. Humble, Scaling quantum approximate optimization on near-Term hardware, *Sci. Rep.* 12 (1) (2022) 12388. <https://doi.org/10.1038/s41598-022-14767-w>
- [42] A. Pellow-Jarman, S. McFarthing, I. Sinayskiy, D.K. Park, A. Pillay, F. Petruccione, The effect of classical optimizers and ansatz depth on QAOA performance in noisy devices, *Sci. Rep.* 14 (2024) 16011. <https://doi.org/10.1038/s41598-024-66625-6>